The Journal of Open Source Software

Pythagora: A Python Package for Modeling the Impact of Social Networks on Market Outcomes

Sarah T. Wilker [1*] **and Annie K. Lamar** [2*]

1 Thomas F. Cooper Post-doctoral Fellow and Visiting Assistant Professor of Classics, Oberlin College, USA **2** Assistant Professor of Computational Classics, University of California, Santa Barbara, USA ***** These authors contributed equally.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Sebastian Benthall

Reviewers:

- @dataspider
- @pitmonticone

Submitted: 21 August 2024
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

The word *agora* is an ancient Greek word meaning "marketplace." In the ancient Greek world, however, the *agora* was more than a market. The *agora* also served as a place for gathering with one's community, catching up on the news, and participating in democratic processes. `Pythagora` (**pythagora**) allows you to model the impact of these varied and vibrant everyday exchanges on supply and demand in the marketplace by introducing community interactions to market simulations.

`Pythagora` is an agent-based modeling package that allows for the construction of experiments that test the impact of select types of social networks on production and consumption outcomes. Specifically, `Pythagora` simulates how individuals (agents) in small-world social networks and scale-free social networks (as well as control versions of these networks) buy and sell different items based on interactions within their social networks.

Agents in `Pythagora` have complex social lives: they belong to communities and are connected in social networks both inside and outside those communities. The purpose of this multidimensional social landscape is to test the impact of different types of social relationships on what individual agents buy and sell, and how these individual transactions build over time to create long-term production and consumption trends.

Statement of need

`Pythagora` is the first software package in Python (to our knowledge) to introduce the concept of community-based incentives to agent-based market models. This is a new and exciting contribution to the field of social network analysis; users can now run experiments and market simulations that consider the influence of social networks on buyer purchase decisions and market supply.

One major advantage of `Pythagora` for users is its ease of use and installation. Many other agent-based modeling environments (e.g. Netlogo (Wilensky, 1999)) require standalone software downloads or the use of a web-based platform with limited capability; this restricts researchers' ability to change parameters, design custom experiments, and perform novel data analyses. `Pythagora` also relies heavily on and interfaces well with the existing functions in the `NetworkX` Python package (Hagberg et al., 2008), making integrating experiments from `Pythagora` into an existing research pipeline a streamlined process.

Finally, `Pythagora` allows users to run experiments with a single method call, increasing accessibility for researchers without substantial coding experience. The results of experiments are stored in .csv files, making it easy for researchers to load results into a Pandas Dataframe or Excel, depending on their research team's skills and preferences.

Pythagora was designed to be used by researchers in a variety of fields, including and especially economics, cultural analytics, historical sciences, archaeology, or network science. An early version of this package was implemented in a recent dissertation from Stanford University (Wilker, 2023). The current version of Pythagora was also cited in a forthcoming (expected 2024) article in the *Journal on Hellenistic and Roman Material Culture* (Wilker, n.d.).

Installation & Documentation

To install *Pythagora*, you can use pip:

```
pip install pythagora
```

Pythagora also requires NetworkX (3.2), which can be installed by running this command:

```
pip install networkx==3.2
```

A full API documentation is available at stwilker.github.io/pythagora. Users will find the documentation on experiment types and changeable parameters in the `experiments.py` module here.

To submit a feature request or report an issue, users should use GitHub's issue tracker.

Getting Started

Pythagora allows you to customize your agent-based market simulations in several ways. There are two general network protocols to choose from: small-world and scale-free. Within each type of algorithm there are several adjustable parameters; detailed information about each parameter is available in the documentation.

The experiments class is the only class with which most users need to interact. You can change all experimental parameters in the main experiment function. The **threshold** values define at what points buyers decide to purchase (or not purchase) particular items available in the marketplace.

Every simulation will include agents (buyers and sellers), communities (groups to which buyers belong), and a market (groups of communities and items for sale). An **epoch** is a single market cycle; buyers enter the market with a purchasing intention, make their purchase, and revise their future purchase intentions. For analysis purposes, changes to the network and community structures are recorded as **time steps** in the data logs.

Small-World Simulation

The small-world simulation contains all the methods you will use to model how consumption practices change among communities of buyers as they move in and out of being in a small-world network. In practice, this means that buyers purchasing items go from being minimally connected (long path lengths and high clustering for individual communities) to being maximally connected (short path lengths and low clustering for individual communities).

Here is an example of how to run a small-world simulation:

```
from pythagora import experiments

experiments.small_world_simulation (number_of_buyers=100,
                                    minimum_number_of_communities=3,
                                    minimum_community_fill=3,
                                    assemblage=['Item1', 'Item2', 'Item3', 'Item4'],
                                    number_of_epochs=100,
                                    upper_threshold=.66,
```

DRAFT

```
                                        lower_threshold=.33,
                                        death_threshold=0.01,
                                        results_directory='results/',
                                        probability_of_rewire=0.5,
                                        community_start_structure="dense",
                                        verbose=True)
```

This method call will run a small-world experiment for 800 epochs. The results of the experiment will be placed in the directory indicated by the `results_directory` parameter.

Scale-Free Simulation

The scale-free simulation contains all the methods you will use to model how consumption practices change among communities of buyers as they join a scale-free social network (i.e., a network that grows through preferential attachment) and this network determines their purchasing behavior in the market space. A novel feature of `Pythagora` is the implementation of community bonuses; this allows the user to define how much influence community affiliation has on buyer purchase intention.

Here is an example of how to run a scale-free simulation:

```
from pythagora import experiments

experiments.scale_free_simulation(number_of_buyers=50,
                                  minimum_number_of_communities=5,
                                  minimum_community_fill=3,
                                  assemblage=['Item1', 'Item2', 'Item3', 'Item4'],
                                  number_of_epochs=100,
                                  upper_threshold=.66,
                                  lower_threshold=.33,
                                  death_threshold=0.01,
                                  results_directory='results/',
                                  initial_set_size=15,
                                  set_size=5,
                                  community_bonus=1,
                                  verbose=True)
```

This method call will run a small world experiment for 800 epochs. The results of the experiment will be placed in the directory indicated by the `results_directory` parameter.

Experimental Controls and Random Simulations

You can also run control experiments for both small-world and scale-free network protocols. Here is an example of how to run a control simulation for a small-world network, which is initialized with a random graph rather than a predefined network structure.

```
from pythagora import experiments

experiments.small_world_control(number_of_buyers=50,
                                minimum_number_of_communities=3,
                                minimum_community_fill=3,
                                assemblage=['Item1', 'Item2', 'Item3', 'Item4'],
                                number_of_epochs=100,
                                upper_threshold=.66,
                                lower_threshold=.33,
                                death_threshold=0.01,
                                results_directory='results/',
                                rewire_prob=0.5,
```

DRAFT

```
                                    link_prob=0.5,
                                    verbose=True)
```

Software History & Author Contributions

`Pythagora` has been in development since 2022. It was first used to generate experimental data for Sarah T. Wilker's doctoral dissertation at Stanford University (Wilker, 2023). `Pythagora` has been significantly expanded over the past two years, including a complete refactoring of the experimental pipeline.

The below author contribution statement was developed based on the CRediT (Contributor Roles Taxonomy) framework.

S.T. Wilker: Conceptualization, Methodology, Software, Validation, Formal Analysis, Investigation, Data Curation, Writing - Original Draft, Writing - Review & Editing; **A.K. Lamar**: Software, Validation, Formal Analysis, Data Curation, Writing - Review & Editing, Visualization

References

Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, 11–15.

Wilensky, U. (1999). *Netlogo*. Evanston, IL. https://ccl.northwestern.edu/netlogo

Wilker, S. T. (n.d.). Toasting across empires: Drinking vessels, production networks, and consumption communities in the hellenistic–early roman aegean and eastern mediterranean. *Journal on Hellenistic and Roman Material Culture*, *13*.

Wilker, S. T. (2023). *The social life of ancient markets: Using formal network approaches and ceramic data to reconceptualize market behavior in the late classical–early hellenistic (400–200 BCE) southeast aegean* [Dissertation]. Stanford University.

DRAFT